PROJET ALGORITHMIQUE

PROJET :

Ce projet vise à modéliser la gestion des patients dans un centre de santé. Chaque patient est décrit par un ensemble d'informations médicales et administratives. Les patients sont stockés dans un tableau non plein, ce qui oblige l'étudiant à gérer dynamiquement les insertions, suppressions, tris et recherches.

AUTEURS GOUPE 3 :

- Cheikh Magueye Thiam
- Diarra Mane
- Mouhamadou Mansour Niang
- Amadou Tidiane Sao

Table des matières

Libellé du projet : 3
I- Partie 1 : Déclaration 3
Explication : 3
II-Partie 2 : Saisie Ajout et Affichage des patients 4
Explication : 4
Explication : 5
III-Partie 3 : Recherche et trie 5
Explication : 5
IV-Partie 4 : Traitements avancés 7
Explication : 7
V-Partie 5 : Récursivité & trichotomie 8
Explication : 8
VI-Partie 6 : Menu principal 10

Libellé du projet :

Concevoir un **programme algorithmique** permettant de gérer les **patients d'un centre de santé** à l'aide d'**enregistrements**, de **tableaux non pleins**, et de **procédures/fonctions**, en intégrant des **algorithmes de tri et de recherche** (itératifs et récursifs)

I- Partie 1 : Déclaration

Explication :

Cette partie définit les structures de données utilisées dans tout le programme. L'enregistrement Patient regroupe toutes les informations liées à un patient. Le tableau non plein permet de stocker dynamiquement les patients grâce au compteur nb.

1-Déclarer le **type enregistrement** Patient :

❖ Code Algo Box:

```
TYPE Patient
  id EST_UN entier
  nom EST_UN chaine
  prenom EST_UN chaine
  age EST_UN entier
  diagnostic EST_UN chaine
  etat EST_UN chaine
FIN_TYPE
```

2-Déclarer le **tableau non plein** et les variables nécessaires :

❖ Code Algo Box:

```
N <- 100
T EST_UN tableau [1..N] DE Patient
nb <- 0
```

II-Partie 2 : Saisie Ajout et Affichage des patients

Explication :

Cette partie assure la consultation des données. Elle permet soit l'affichage d'un seul patient, soit de tous les patients enregistrés.

3- **Procédure SaisirPatient** permettant de saisir les informations d'un patient :

❖ Code Algo Box:

```
PROCEDURE SaisirPatient(VAR p : Patient)
  LIRE p.id
  LIRE p.nom
  LIRE p.prenom
  LIRE p.age
  LIRE p.diagnostic
  LIRE p.etat
   FIN_PROCEDURE
```

4-**Procédure AjouterPatient** qui ajoute un patient au tableau :

❖ Code Algo Box:

```
 PROCEDURE AjouterPatient()
 SI nb < N ALORS
   nb <- nb + 1
   SaisirPatient(T[nb])
 SINON
   AFFICHER "Tableau plein"
 FIN_SI
  FIN_PROCEDURE
```

Explication :

Cette partie assure la consultation des données. Elle permet soit l'affichage d'un seul patient, soit de tous les patients enregistrés.

5- Procédure Affichage Patient :

❖ Code Algo Box:

```
PROCEDURE AfficherPatient (p: Patient)
  AFFICHER p.id, p.nom, p. prenom, p.age, p. diagnostic, p. etat
FIN_PROCEDURE
```

6-Procédure Affichage Tous les Patients :

❖ Code Algo Box:

```
PROCEDURE AfficherTousPatients ()
  POUR i DE 1 A nb FAIRE
    AfficherPatient(T[i])
  FIN_POUR
FIN_PROCEDURE
```

III-Partie 3 : Recherche et trie

Explication :

Les algorithmes de recherche permettent de localiser rapidement un patient. Le tri à bulles est utilisé pour ordonner les patients par identifiant, condition nécessaire pour la recherche dichotomique.

7- Fonction de recherche séquentielle d'un patient par id :

❖ Code Algo Box:

```
FONCTION RechercheSeq (idrech: entier): entier
  POUR i DE 1 A nb FAIRE
    SI T[i].id = idrech ALORS
      RETOURNER i
    FIN_SI
  FIN_POUR
  RETOURNER -1
FIN_FONCTION
```

8-Trie des patients par ordre croissant de l'id (tri à bulles amélioré) :

❖ Code Algo Box:

```
PROCEDURE TriBulles ()
    echange <- VRAI
TANT_QUE echange FAIRE
    echange <- FAUX
POUR i DE 1 A nb-1 FAIRE
SI T[i].id > T[i+1].id ALORS
    temp <- T[i]
T[i] <- T[i+1]
T[i+1] <- temp
    echange <- VRAI
FIN_SI
FIN_POUR
FIN_TANT_QUE
FIN_PROCEDURE
```

9- Fonction de recherche dichotomique itérative par id :

❖ Code Algo Box:

```
FONCTION RechercheDicho(idrech : entier) : entier
debut <- 1
fin <- nb
TANT_QUE debut <= fin FAIRE
milieu <- (debut + fin) DIV 2
SI T[milieu].id = idrech ALORS
RETOURNER milieu
SINON SI idrech < T[milieu].id ALORS
fin <- milieu - 1
SINON
debut <- milieu + 1
FIN_SI
FIN_TANT_QUE
RETOURNER -1
FIN_FONCTION
```

10-Écrire une fonction récursive de recherche dichotomique **:**

❖ Code Algo Box:

```
FONCTION RechercheDichoRec(idrech : entier, debut : entier, fin : entier) :
entier
  SI debut > fin ALORS
    RETOURNER -1
  SINON
    milieu <- (debut + fin) DIV 2

    SI T[milieu].id = idrech ALORS
      RETOURNER milieu
    SINON SI idrech < T[milieu].id ALORS
      RETOURNER RechercheDichoRec(idrech, debut, milieu - 1)
    SINON
      RETOURNER RechercheDichoRec(idrech, milieu + 1, fin)
    FIN_SI
  FIN_SI
FIN_FONCTION
```

IV-Partie 4 : Traitements avancés

Explication :

Ces traitements permettent la maintenance des données : suppression, modification et filtrage des patients hospitalisés.

11- Supprimer un patient à partir de son id :

❖ Code Algo Box:

```
PROCEDURE SupprimerPatient (idrech : entier)

  pos <- RechercheSeq(idrech)

  SI pos <> -1 ALORS

    POUR i DE pos A nb-1 FAIRE

      T[i] <- T[i+1]

    FIN_POUR

    nb <- nb - 1

  FIN_SI

FIN_PROCEDURE
```

12-Modifier le diagnostic d'un patient.

❖ Code Algo Box:

```
PROCEDURE ModifierDiagnostic(idrech : entier)
  pos <- RechercheSeq(idrech)
  SI pos <> -1 ALORS
    LIRE T[pos].diagnostic
  FIN_SI
FIN_PROCEDURE
```

13-Afficher uniquement les patients hospitalisés.

❖ Code Algo Box:

```
PROCEDURE AfficherHospitalises ()

POUR i DE 1 A nb FAIRE

SI T[i].etat = "hospitalisé" ALORS

AfficherPatient(T[i])

FIN_SI

FIN_POUR

FIN_PROCEDURE
```

V-Partie 5 : Récursivité & trichotomie

Explication :

La **recherche trichotomique** consiste à diviser le tableau **trié** en **trois parties** au lieu de deux, puis à poursuivre récursivement la recherche dans la partie pertinente.

14- fonction récursive de recherche trichotomique par id.

❖ Code Algo Box:

```
FONCTION RechercheTrichoRec(idrech : entier, debut : entier, fin : entier) : entier
  SI debut > fin ALORS
    RETOURNER -1
  SINON
    tiers1 <- debut + (fin - debut) DIV 3
    tiers2 <- fin - (fin - debut) DIV 3

    SI T[tiers1].id = idrech ALORS
      RETOURNER tiers1
    SINON SI T[tiers2].id = idrech ALORS
      RETOURNER tiers2
    SINON SI idrech < T[tiers1].id ALORS
      RETOURNER RechercheTrichoRec(idrech, debut, tiers1 - 1)
    SINON SI idrech > T[tiers2].id ALORS
      RETOURNER RechercheTrichoRec(idrech, tiers2 + 1, fin)
    SINON
      RETOURNER RechercheTrichoRec(idrech, tiers1 + 1, tiers2 - 1)
    FIN_SI
  FIN_SI
FIN_FONCTION


pos <- RechercheTrichoRec(id, 1, nb)

SI pos <> -1 ALORS
  AfficherPatient(T[pos])
SINON
  AFFICHER "Patient introuvable"
FIN_SI
```

15-Comparaison :

La recherche dichotomique divise le tableau en deux parties et est plus simple à implémenter, tandis que la recherche trichotomique divise le tableau en trois parties, ce qui réduit légèrement le nombre d'itérations mais augmente la complexité de l'algorithme.

VI-Partie 6 : Menu principal

16- Concevoir un menu interactif permettant :

```
DEBUT_ALGORITHME

REPETER
   AFFICHER "===== MENU GESTION DES PATIENTS ====="
   AFFICHER "1 - Ajouter un patient"
   AFFICHER "2 - Afficher tous les patients"
   AFFICHER "3 - Rechercher un patient"
   AFFICHER "4 - Trier les patients par ID"
   AFFICHER "5 - Supprimer un patient"
   AFFICHER "6 - Modifier le diagnostic d'un patient"
   AFFICHER "7 - Quitter"
   AFFICHER "Votre choix : "
   LIRE choix

   SELON choix FAIRE

      1:
         AjouterPatient()

      2:
         SI nb = 0 ALORS
            AFFICHER "Aucun patient enregistré"
         SINON
```

```
22            AFFICHER "Aucun patient enregistré"
23         SINON
24            AfficherTousPatients()
25         FIN_SI
26
27      3:
28         AFFICHER "Entrer l'ID du patient à rechercher : "
29         LIRE idrech
30         pos <- RechercheSeq(idrech)
31         SI pos <> -1 ALORS
32            AFFICHER "Patient trouvé :"
33            AfficherPatient(T[pos])
34         SINON
35            AFFICHER "Patient introuvable"
36         FIN_SI
37
38      4:
39         SI nb > 1 ALORS
40            TriBulles()
41            AFFICHER "Patients triés par ID"
42         SINON
43            AFFICHER "Tri impossible"
44         FIN_SI
```

```
45
46      5:
47         AFFICHER "Entrer l'ID du patient à supprimer : "
48         LIRE idrech
49         SupprimerPatient(idrech)
50
51      6:
52         AFFICHER "Entrer l'ID du patient à modifier : "
53         LIRE idrech
54         ModifierDiagnostic(idrech)
55
56      7:
57         AFFICHER "Fin du programme"
58
59      AUTRE:
60         AFFICHER "Choix invalide"
61
62   FIN_SELON
63
64 JUSQU_A choix = 7
65
66 FIN_ALGORITHME
67
```